**WPI**

**WORCESTER POLYTECHNIC INSTITUTE**  
**ROBOTICS ENGINEERING PROGRAM**

# **Lab 3 – Inverse Kinematics and Trajectory Generation**

**SUBMITTED BY**  
**Sukriti Kushwaha**  
**Istan Slamet**  
**Tracy Yang**

Date Submitted: September 21, 2023  
Date Completed: September 21, 2023  
Course Instructor: Dr. Agheli  
Lab Section: RBE 3001 A'12

## **Abstract**

This report overviews the experiments performed in Lab 3, which was used to learn how to utilize inverse kinematics and trajectory planning to move an OpenManipulator-X robotic arm. The experiments primarily included analysis of joint and end-effector positions, velocities, and accelerations in both the joint and task spaces using both cubic and quintic polynomial trajectory methods. All experiments were conducted using MATLAB.

## Introduction

The purpose of Lab 3 was to formulate the inverse kinematics of the OpenManipulator-X robot arm and perform trajectory planning in joint and task space using cubic polynomials and linear interpolation. Inverse kinematics was used to find the angles of each of the joints with respect to their home position, given the end-effector's pose. The inverse kinematics equations and resulting values were also used for the path planning. Using trajectory planning combined with inverse kinematics, we were able to achieve smooth motion planning between vertices of a triangle in the task space of the robot.

## Methodology

From following the procedure of the lab, we started by calculating the inverse kinematics of the OpenManipulator-X arm, specifically finding the four joint parameters  $q_1$ ,  $q_2$ ,  $q_3$ , and  $q_4$ , given the pose of the end-effector with respect to the horizontal plane ( $x$ ,  $y$ ,  $z$ ,  $\alpha$ ) in the task space.

For number 1, we began by calculating the value of  $\theta_1$ . Figure \_\_\_ is a representation of the arm and angle location, as well as a set of calculations used to determine the angle value. We determined  $r$ , the distance from the base joint to the end-effector by taking the square root of the given  $x$  and  $y$  squared values' sum. From there, we used  $\arctan2$  to calculate the value of  $\theta_1$ . To find the values of  $\theta_2$ ,  $\theta_3$ , and  $\theta_4$ , we used the 2-link planar method. Figure \_\_\_ shows a graphic representation of the arm and its angle locations as well as a set of calculations to determine the angle values. Because the arm has three links, we had to reference the position of joint 4 to find  $\theta_2$  and  $\theta_3$ . When we determined this, we found these values using the law of cosines in combination with  $\arctan2$ . Once we determined these values, we found  $\theta_4$  by subtracting  $\theta_2$  and  $\theta_3$  from the given  $\alpha$  value, because the sum of  $\theta_2$ ,  $\theta_3$ , and  $\theta_4$  equals  $\alpha$ .

After calculating the robot's inverse kinematics, we started programming the function `ik3001()` for part 2. `ik3001()` uses the end-effector position as input and outputs the corresponding joint space angles ( $q_1$ ,  $q_2$ ,  $q_3$ , and  $q_4$ ). We transferred our calculations from part 1 into MATLAB. Initially, our original calculations did not work, however, and we tried several different methods. We were finally able to get this working when we changed where we were referencing  $\theta_2$  from and used the law of sines to solve for one of the intermediate angles,  $\beta$ . Instead of referencing  $\theta_2$  about the  $x$ -axis, we referenced it about the  $z$ -axis, subtracting the intermediate angles and joint offset from 90. We performed a similar calculation for  $\theta_3$ . And  $\theta_4$  was calculated by subtracting  $\theta_2$  and  $\theta_3$  from the given  $\alpha$ .

To test `ik3001()`, we created a separate script, `lab3`, which we used to compare the outputs of `fk3001()` with `ik3001()` because the two functions have the exact opposite inputs and outputs. We used `fk3001()` for the home position and three other arbitrary positions. Since `fk3001()` outputs an end-effector position, we input that value into `ik3001()` and compared the value. These tests were sufficient for sign-off #1.

For part 3, we needed to test the validity of `ik3001()` even further by using the function to move the robot in a triangle and analyze several plots generated from the movement. We created a 3D stick model plot using `plot_arm()`, plotting the robot's live movement. We also 2D-plotted the measured  $x$ ,  $y$ ,  $z$  positions of the end-effector over time as well as a 3D plot of the measured path. Finally, we created a plot with

four lines corresponding to each of the joint angles over time. The results we obtained were sufficient for sign-off #2.

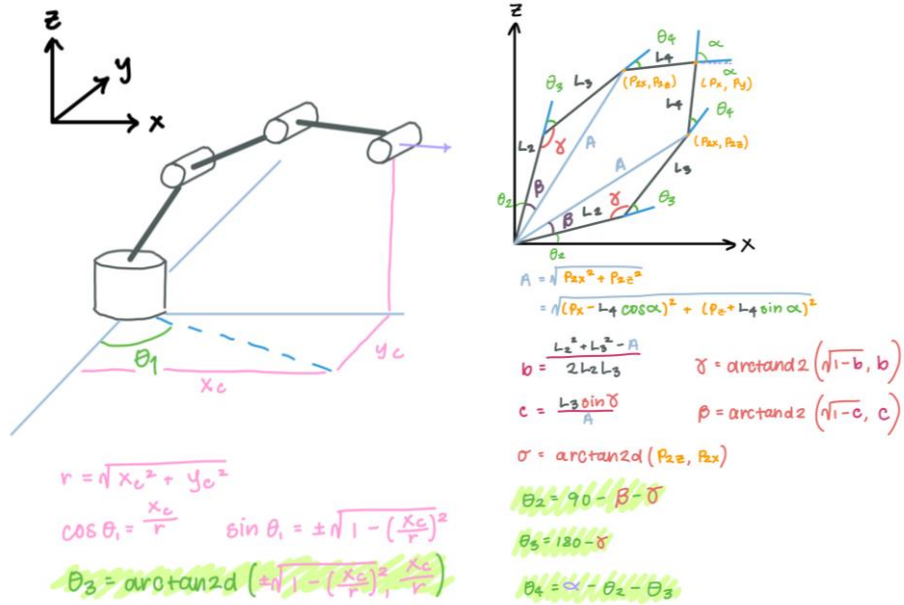
For part 4, we began the trajectory planning by creating the 'Traj\_Planner.m' class and the function cubic\_traj() within it, which solves for a cubic 3rd-order polynomial trajectory between two via points. It takes in the starting and ending times, velocities, and ending positions, and outputs a 4x1 matrix containing the coefficients of the polynomial. We substituted the time inputs into the matrix form of the cubic and divided this matrix by a column vector of the input joint positions and velocities. This results in the coefficients of the polynomial.

For part 5, we started by creating the run\_trajectory() function in Robot.m which lets the arm follow trajectories. The function takes in two arguments – the trajectory coefficients generated from cubic\_traj(), and the total amount of time the trajectory should take. From there, run\_trajectory() moves the servo to the set task space points as well as collects the measured joint angles and the time taken to go to the goal point. We tested this function by having it draw a triangle, which needed us to run it three times for each side.

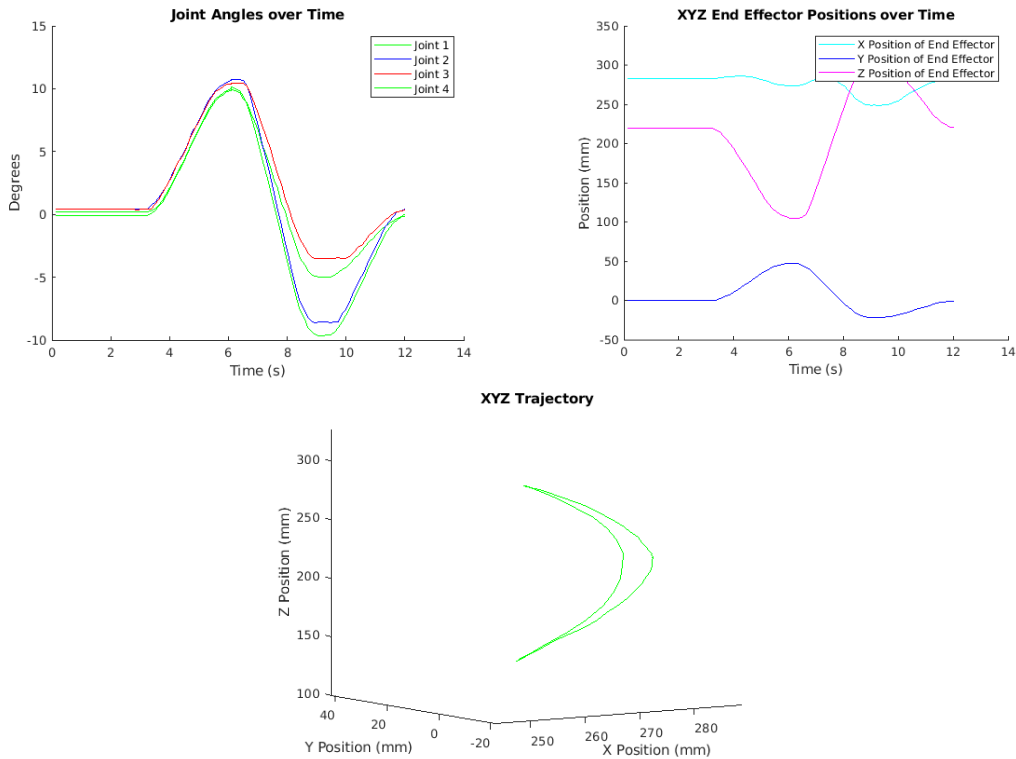
For part 6, we expanded run\_trajectory() to work with coefficients given from the task space. We introduced a new input argument, GETTASK, to determine if they are from joint or task space. If the coefficients are from task space, we run the coefficients through ik3001() before sending that output to servo\_jp() to move the robot across the trajectory. We also created plots for the joint and end-effector positions, angles, and acceleration over time.

For part 7, we created a new function quintic\_traj() within Traj\_Planner.m to solve for a cubic 5th-order polynomial trajectory between two via points. It takes in the same arguments as cubic\_traj() with the addition of the starting and ending acceleration values. It outputs a 6x1 matrix containing the fifth-order polynomial coefficients. To test quintic\_traj(), we ran the same triangle coordinates and compared its motion with the cubic\_traj() motion. We also plotted the joint and end-effector positions, velocities, accelerations, and angles over time.

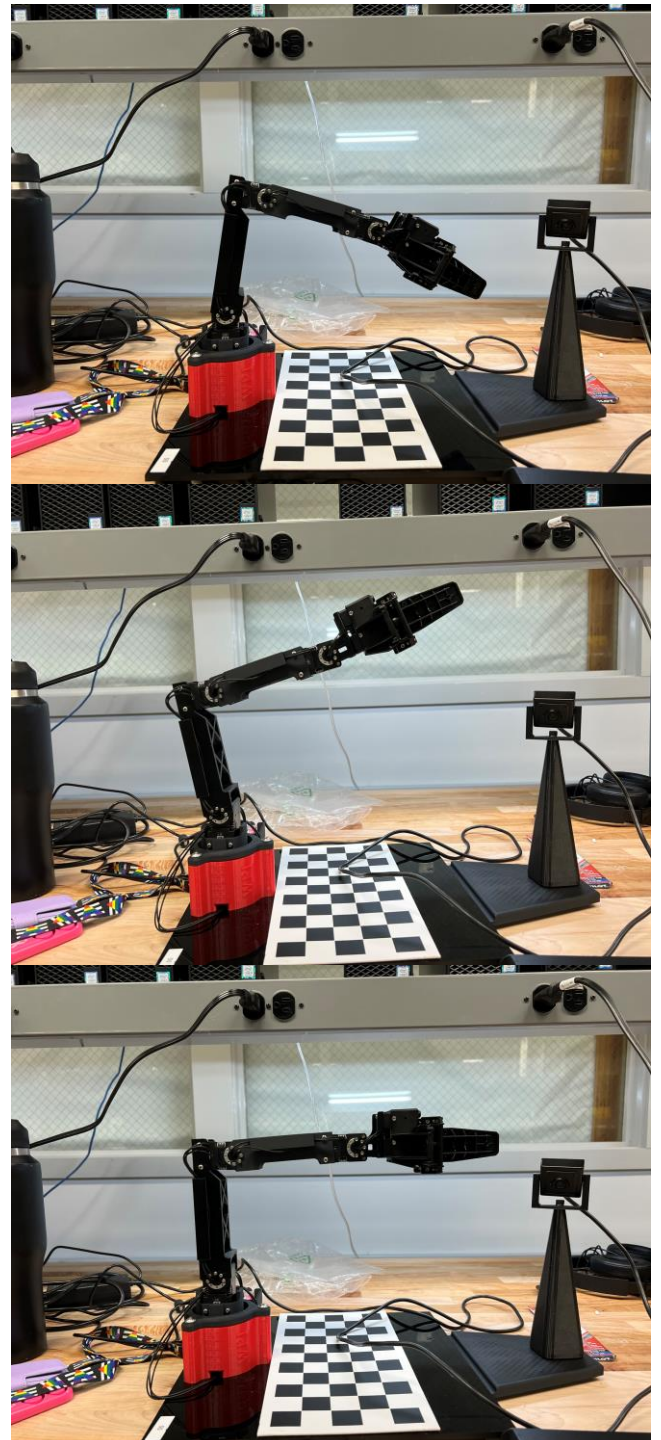
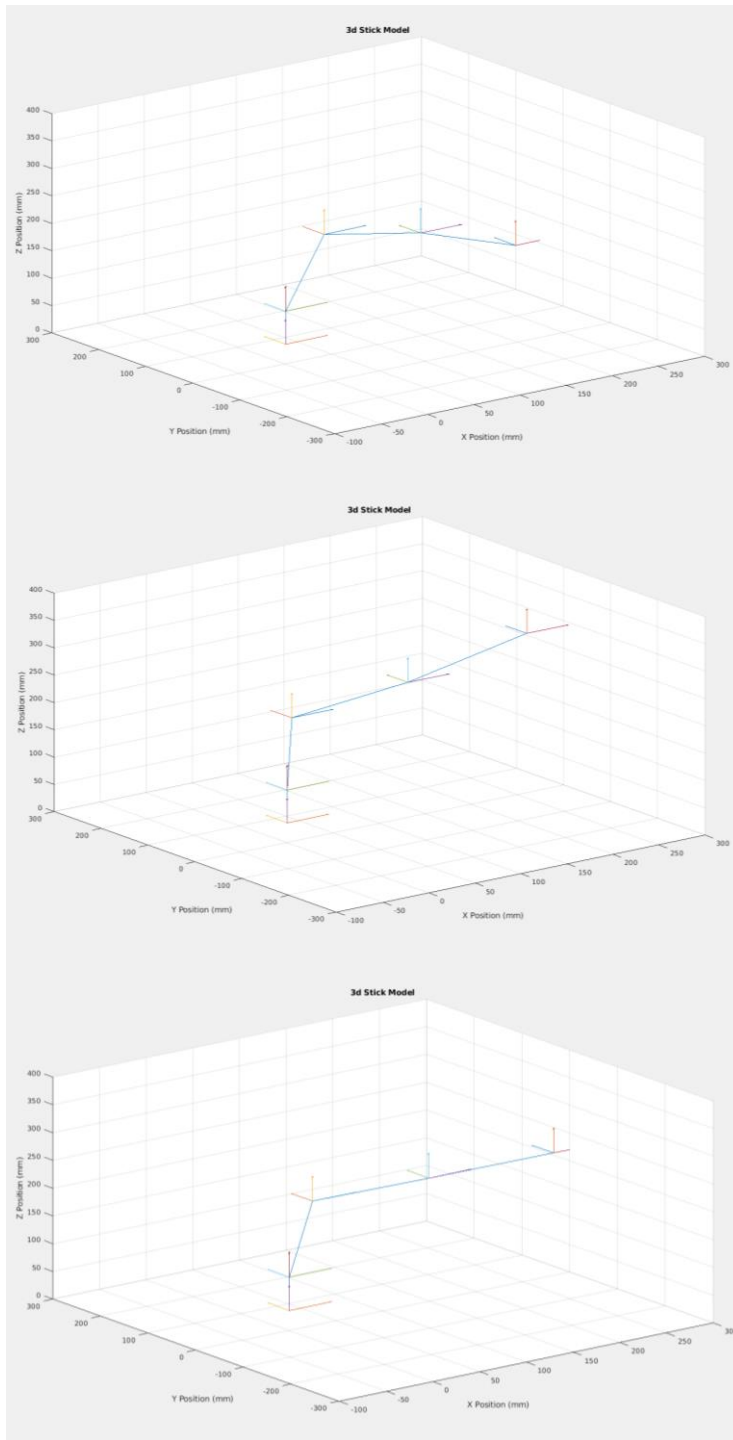
## Results



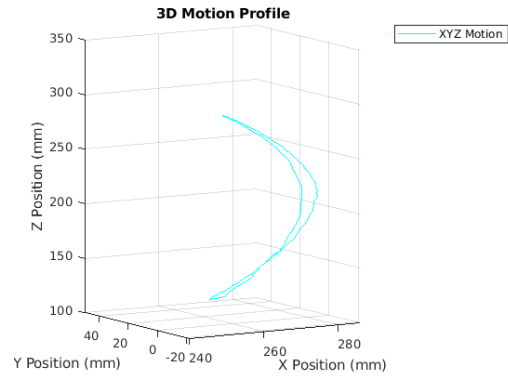
Figures 1 and 2: Drawings and calculations of the robot arm's inverse kinematics



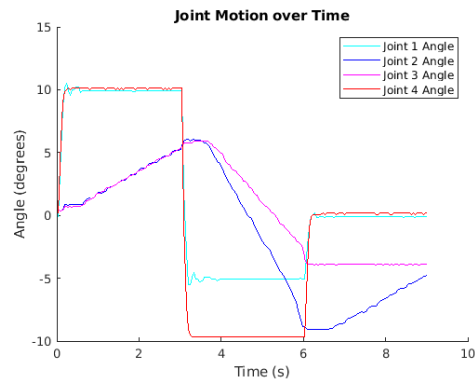
Figures 3, 4, and 5: From the top left to the bottom right, these figures depict the distance that joints 1-3 take in the joint space, the position of the x, y, and z of the end effector in the task space, and the end effector's trajectory



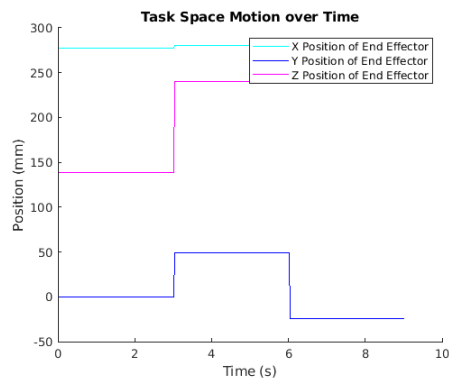
**Figures 6, 7, 8, 9, 10, 11, and 12: On the left side, the figures depict the 3D stick models of each vertex when the robot is going in a triangle-shaped trajectory. On the right is the picture of the corresponding positions**



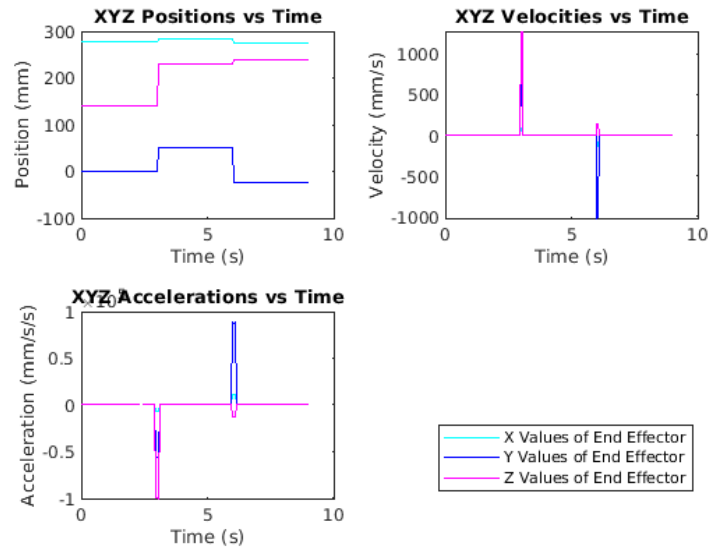
**Figure 13: The depiction of the 3D motion profile of the cubic trajectory pathing**



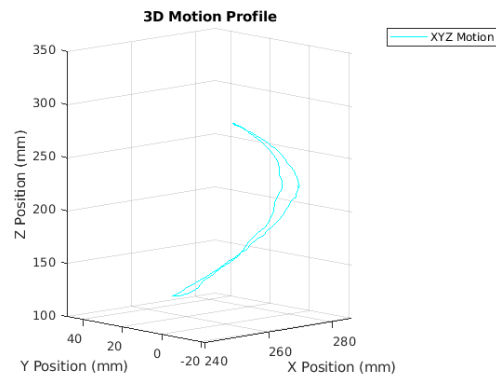
**Figure 14: The depiction of the joint motion of each joint over time when using cubic trajectory planning**



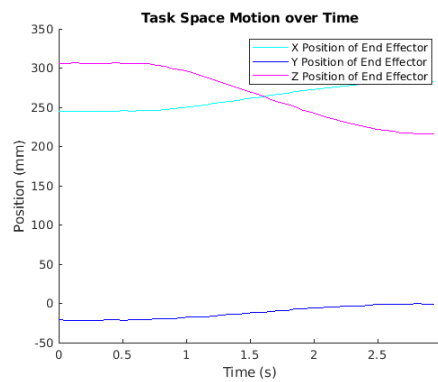
**Figure 15: The depiction of the task space motion in the x, y, and z directions of the end effector when using cubic trajectory planning**



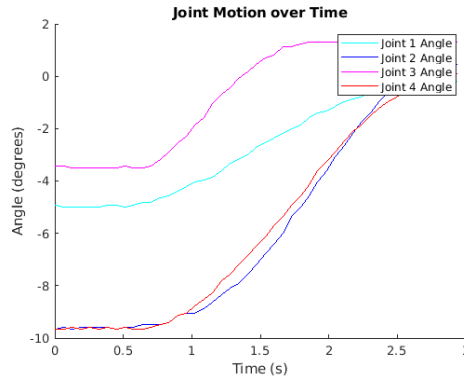
**Figure 16: The values when using cubic trajectory of the x, y, and z values of the end effector in relation to time, where the top left is the positions, the top right is the velocities, and the bottom left is the accelerations.**



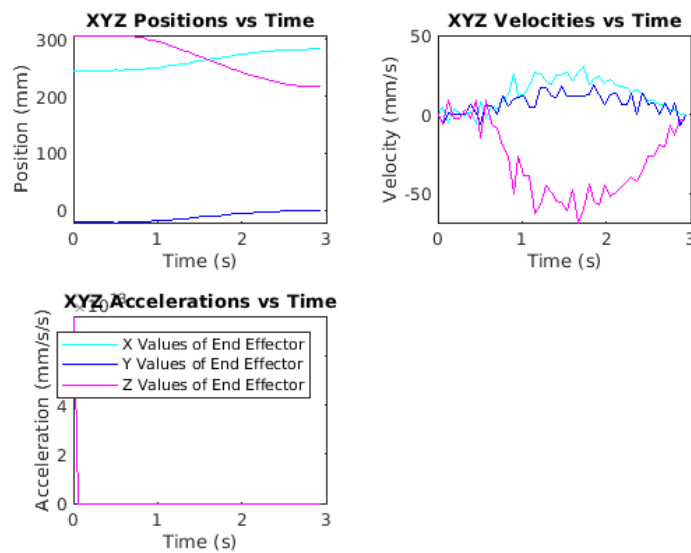
**Figure 17: The depiction of the 3D motion profile of the joint space quintic trajectory pathing**



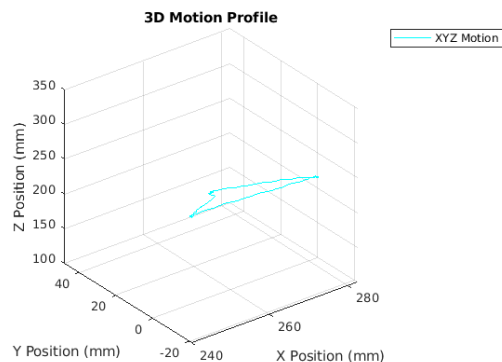
**Figure 18: The depiction of the joint motion of each joint over time when using joint space quintic trajectory planning**



**Figure 19: The depiction of the task space motion in the x, y, and z directions of the end effector when using joint space quintic trajectory planning**

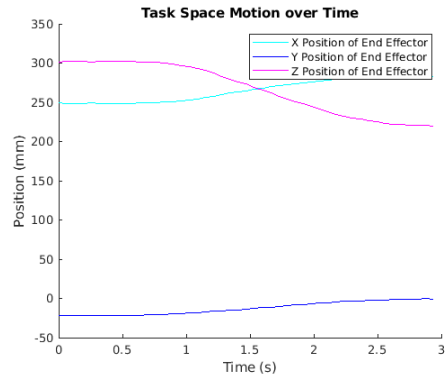


**Figure 20: The values when using joint space quintic trajectory of the x, y, and z values of the end effector planning in relation to time, where the top left is the positions, the top right is the velocities, and the bottom left is the accelerations**

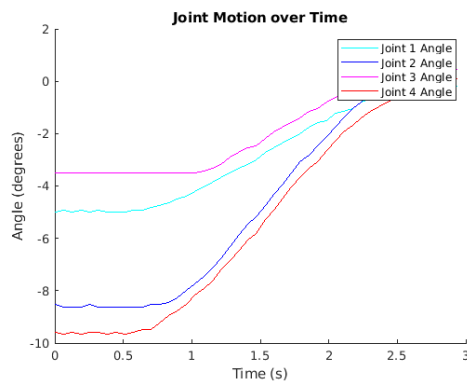


**Figure 21: The depiction of the 3D motion profile of the task space quintic trajectory pathing**

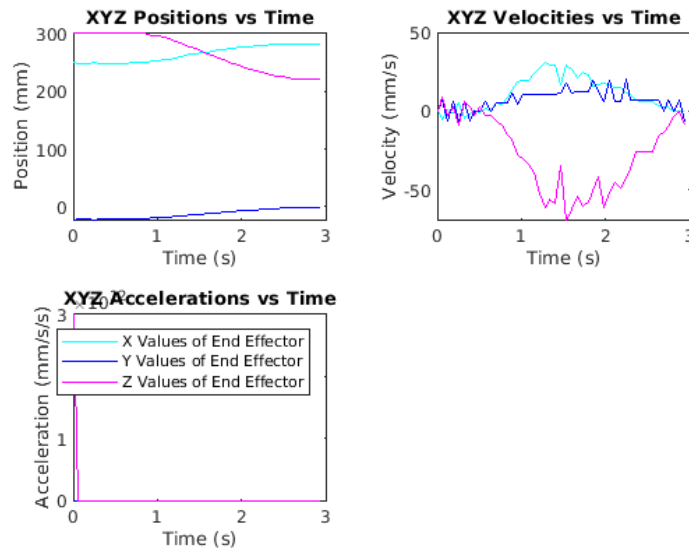




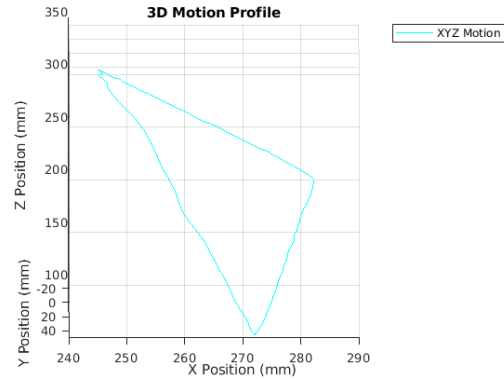
**Figure 22: The depiction of the task motion of each joint over time when using joint space quintic trajectory planning**



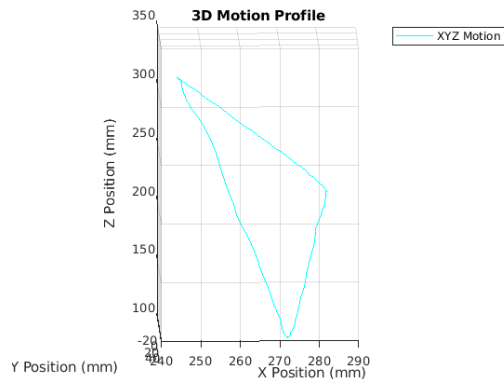
**Figure 19: The depiction of the joint space motion in the x, y, and z directions of the end effector when using joint space quintic trajectory planning**



**Figure 24: The values when using task space quintic trajectory of the x, y, and z values of the end effector planning in relation to time, where the top left is the positions, the top right is the velocities, and the bottom left is the accelerations**



**Figure 25: The cubic trajectory path of the task space when the triangle is normal to the plane**



**Figure 26: The quintic trajectory path of task space when the triangle is normal to the plane**

## Discussion

We ensured that our calculations for inverse kinematics were correct and that we were referencing points, link lengths, and angles correctly in relation to how the robot was positioned, the derivations of which can be found in Figures 1 and 2. We then translated these calculations into MATLAB, where we tested the function `ik3001()` by reverse engineering the angles. Through `fk3001()` we solved for the position matrix to find positions that we knew. These positions were inputted into the `ik3001()`, and we looked to see if the joint values output were the same as the initial joint values that we put in `fk3001()`. With this, we confirmed the function was correct.

From there, we put the function to the test by forming a triangle in the workspace of the robot and plotted the resulting motion for parts 2 and 3. We moved the joints from the home position to the angles of (10, 10, 10, 10), then (-5, -10, -5, -10), and back to the home position. The 3D plots of the motion can be found in Figures 6 through 12. Figure 14 displays a plot of the joint angles over time. In this plot, we can see that all the joints move to the corresponding positions. Figure 15 shows the end-effector task space positions over time. Expectedly, the z position changes the most because the motion causes the most motion in that axis. Figure 13 shows the xyz trajectory of the end-effector in the task space. The plot displays that the motion is not perfectly triangular, but it does display that a closed, three-sided motion is made. And lastly, Figure 16 shows the position, velocity, and acceleration of the xyz end effector values over time.

For parts 5 and 6, we had the robot do the same triangle movement but by using the `run_trajectory()` function. The function creates via-points for the robot to travel to between each movement to make it easier to create trajectories between two input end-effector positions. We plotted the xyz positions, velocities, and accelerations of the joints and end-effector in both the joint space and the task space. The task and joint space motion for cubic overtime can be found in Figures 18 and 19. Comparing the 3D motion of the joint space and task space motions, we found that the task space motion is much more linear and triangle-shaped than the joint space motion. This can be attributed to the fact that the joint space coefficients aim for the shortest path between two angles, causing a curved motion. The task space coefficients aim for the shortest path between two end-effector positions, which causes a linear motion.

For part 7, we performed the same process as in parts 5 and 6, but instead used the `quintic_traj()` function to send values to `run_trajectory()`. The plots for the quintic motion can be found in Figures 17 to 24, which demonstrate nearly identical movements to those from parts 5 and 6. The difference between the curved joint space movements and the linear task space end-effector movements is for the same reason as the cubic trajectory. The positions, velocities, and accelerations are also identical to the cubic trajectory movements because in both cases the functions were fed the same values. From the graphs, we can conclude that utilizing inverse kinematics for trajectory makes the motion of the arm smoother and more linear when compared to moving in the joint space.

## Conclusion:

Overall, lab 3 allowed us to calculate the inverse kinematics of the OpenManipulator-X's arm and implement the calculations in MATLAB. We performed both cubic and quintic trajectories in the joint and task spaces of the robot arm and plotted the motion between via points in 2D and 3D graphs. We also calculated the velocity and acceleration of the joint positions and created plots of their calculations with respect to time. We found that the task space motion is much more linear and triangle-shaped than the joint space motion, proving how inverse kinematics allow for smooth motion travel.

## Appendix A: Authorship

| Section      | Author           |
|--------------|------------------|
| Introduction | Sukriti Kushwaha |
| Methodology  | Istan Slamet     |
| Results      | Tracy Yang       |
| Discussion   | Everyone :)      |
| Conclusion   | Sukriti Kushwaha |